

Inhaltsverzeichnis

01 Grundlagen

- Programmieren als Denkweise
 - Informatiker-Denkweise
 - Formale vs. natürliche Sprachen
- Arithmetische Operatoren
- Zahlentypen
 - Typkonvertierung
- Operationsreihenfolge (PEMDAS)
- Strings (Zeichenketten)
- Funktionen (Built-in)
- Ausdrücke und Werte
- Häufige Fehler und Debugging
 - Syntaxfehler
 - TypeError
- Prüfungstipps
- Mathematische Umrechnungen (Übungsbeispiele)

02 Variablen & Anweisungen

- Variablen und Zuweisungen
 - Grundlagen
 - Verwendung
 - Regeln für Variablennamen
 - Ungültige Variablennamen
 - Zustandsdiagramme
- Die import-Anweisung
 - math-Modul
 - Punktoperator
- Ausdrücke und Anweisungen
- Die print-Funktion
 - Mehrere Werte ausgeben
- Funktionsargumente
 - Typen und Fehler
- Kommentare
 - Gute Kommentare
- Fehlerarten und Debugging
 - Syntaxfehler
 - Laufzeitfehler
 - Semantische Fehler
- Übungsbeispiele
- Glossar

03 Funktionen

- Funktionen definieren und aufrufen
 - Funktionsdefinition
 - Funktionsaufruf
 - Parameter und Argumente
- Funktionskomposition
 - Aufrufhierarchie
- Wiederholung mit for-Schleifen
- Variablen-Geltungsbereich (Scope)
 - Lokale vs. globale Variablen
- Stack-Diagramme und Debugging
 - Traceback
- Typische Fehler in Funktionen
 - NameError
 - TypeError
 - SyntaxError
- Warum Funktionen?
 - Vorteile
 - Best Practices
- Übungsbeispiele
- Glossar

04 Interfaces & Beispiele

- jupyter-turtle-Modul
 - Grundlegende Befehle
 - Quadrat & Schleifen
- Verkapselung und Verallgemeinerung
 - square, polygon
- Schlüsselwortargumente
- Kreis- und Bogenzeichnung
 - circle, arc, polyline
- Funktionsdesign und Interface
 - Interface vs. Implementierung
- Docstrings
 - Konventionen
- Prä- und Postkonditionen
- Entwicklungsplan
- Refaktorisierung
- Praktische Hilfsfunktionen
 - jump()
- Nützliche Tipps
- Stack-Diagramm
- Glossar

Python Grundlagen

Programmieren als Denkweise

- **Informatiker-Denkweise:** Kombination aus mathematischer Präzision, ingenieurmäßigem Design und wissenschaftlicher Beobachtung
- **Formale vs. natürliche Sprachen:**
 - Formale Sprachen (wie Python): eindeutig, präzise, wörtlich zu nehmen
 - Natürliche Sprachen: mehrdeutig, redundant, enthalten Metaphern und Redewendungen

Arithmetische Operatoren

Operator	Beschreibung	Beispiel	Ergebnis
+	Addition	30 + 12	42
-	Subtraktion	43 - 1	42
*	Multiplikation	6 * 7	42
/	Division (ergibt Fließkommazahl)	84 / 2	42.0
//	Ganzzahl-Division (abrunden)	85 // 2	42
**	Potenzierung	7 ** 2	49
%	Modulo (Rest nach Division)	5 % 2	1

⚠ **Vorsicht:** ^ ist KEIN Potenzierungsoperator (wie in anderen Sprachen), sondern ein bitweiser XOR-Operator

Zahlentypen

Typ	Beschreibung	Beispiel
int	Ganze Zahlen	42
float	Fließkommazahlen	42.0

Typkonvertierung

- `int(42.9)` → 42 (schneidet Dezimalstellen ab)
- `float(42)` → 42.0
- `int("126")` → 126 (wandelt String in Zahl um)
- `float("12.6")` → 12.6

Tipp: Für große Zahlen Unterstriche zur besseren Lesbarkeit verwenden: `1_000_000` statt `1000000`

Operationsreihenfolge (PEMDAS)

1. **Parenteses** (Klammern): `()`

2. **Exponents** (Potenzierung): `**`
3. **Multiplication/Division** (Multiplikation/Division): `*`, `/`, `//`, `%`
4. **Addition/Subtraction** (Addition/Subtraktion): `+`, `-`

Beispiele:

- `6 + 6 ** 2 = 6 + 36 = 42`
- `12 + 5 * 6 = 12 + 30 = 42`
- `(12 + 5) * 6 = 17 * 6 = 102`

Strings (Zeichenketten)

Operation	Beschreibung	Beispiel	Ergebnis
Erstellung	Mit einfachen oder doppelten Anführungszeichen	<code>'Hello'</code> oder <code>"Hello"</code>	<code>'Hello'</code>
<code>+</code>	Verkettung (Concatenation)	<code>'Well, ' + "it's a small" + 'world.'</code>	<code>"Well, it's a small world."</code>
<code>*</code>	Wiederholung	<code>'Spam, ' * 4</code>	<code>'Spam, Spam, Spam, Spam, '</code>
<code>len()</code>	Länge eines Strings	<code>len('Spam')</code>	<code>4</code>

Wichtig:

- Einfache (`'`) und doppelte (`"`) Anführungszeichen sind gleichwertig
- Doppelte Anführungszeichen sind nützlich für Strings mit Apostrophen: `"it's a small"`
- Typografische Anführungszeichen (`" "`) und Backticks (```) sind NICHT erlaubt und erzeugen Syntaxfehler

Funktionen

Funktion	Beschreibung	Beispiel	Ergebnis
<code>round()</code>	Rundet auf die nächste Ganzzahl	<code>round(42.4)</code>	<code>42</code>
		<code>round(42.6)</code>	<code>43</code>
<code>abs()</code>	Absoluter Wert	<code>abs(-42)</code>	<code>42</code>
<code>type()</code>	Gibt den Typ eines Werts zurück	<code>type(42)</code>	<code>int</code>
		<code>type(42.0)</code>	<code>float</code>
		<code>type('Hello')</code>	<code>str</code>
<code>len()</code>	Gibt die Länge eines Strings zurück	<code>len('Hello')</code>	<code>5</code>

Wichtig bei Funktionsaufrufen:

- Runde Klammern (`()`) sind Pflicht!
- `abs(-42)` → korrekt
- `abs -42` → Syntaxfehler

Ausdrücke und Werte

- Ein **Ausdruck** ist eine Kombination aus Operatoren und Werten
- Jeder Ausdruck hat einen **Wert**
- Jeder Wert hat einen **Typ**

Häufige Fehler und Debugging

Syntaxfehler

- Fehlende Klammern bei Funktionsaufrufen: `abs 42` statt `abs(42)`
- Falsche Anführungszeichen: ``Hello`` oder `"Hello"` statt `"Hello"` oder `'Hello'`
- Verwendung von Kommas in großen Zahlen: `1,000,000` erzeugt ein Tupel `(1, 0, 0)` statt einer Zahl

TypeError

- Tritt auf, wenn Operanden den falschen Typ haben
- Beispiel: `'126' / 3` erzeugt einen `TypeError`, weil man einen String nicht durch eine Zahl teilen kann
- Lösung: Typkonvertierung mit `int('126') / 3`

Prüfungstipps

1. **Operatoren vs. Funktionen:** Verstehe den Unterschied zwischen Operatoren (`+`, `-`, `*`, `/`) und Funktionen (`abs()`, `round()`, `len()`)
2. **Typkonvertierung:** Übe die Konvertierung zwischen Datentypen (`int()`, `float()`, `str()`)
3. **Stringoperationen:** Beachte, dass `+` und `*` bei Strings andere Bedeutungen haben als bei Zahlen
4. **Operationsreihenfolge:** Beachte die Priorität von Operatoren (PEMDAS)
5. **Syntaxregeln:** Achte auf korrekte Klammern, Anführungszeichen und Funktionsaufrufe

Mathematische Umrechnungen (Übungsbeispiele)

- Zeit: 1 Minute = 60 Sekunden
- Distanz: 1 Meile $\approx$ 1,61 Kilometer
- Geschwindigkeit: Entfernung $\div$ Zeit

Beispielberechnungen:

1. 42 Minuten und 42 Sekunden in Sekunden:

```
42 * 60 + 42 # = 2520 + 42 = 2562 Sekunden
```

2. 10 Kilometer in Meilen:

```
10 / 1.61 #  $\approx$  6.21 Meilen
```

3. Durchschnittsgeschwindigkeit für 10 km in 42:42 (Minuten: Sekunden):

```
# Zeit in Sekunden
zeit_sek = 42 * 60 + 42 # = 2562 Sekunden

# Distanz in Meilen
distanz_meilen = 10 / 1.61 # ≈ 6.21 Meilen

# Sekunden pro Meile
sek_pro_meile = zeit_sek / distanz_meilen # ≈ 412.56 Sekunden/Meile

# Minuten und Sekunden pro Meile
min_pro_meile = sek_pro_meile // 60 # = 6 Minuten
rest_sek = sek_pro_meile % 60 # ≈ 52.56 Sekunden

# Meilen pro Stunde
mph = distanz_meilen / (zeit_sek / 3600) # ≈ 8.73 mph
```

Python Variablen und Anweisungen

Variablen und Zuweisungen

Grundlagen der Variablen

- Eine **Variable** ist ein Name, der auf einen Wert verweist
- Erstellen einer Variablen mit **Zuweisungsanweisung**: `variable = wert`
- Beispiele:

```
n = 17                # Integer-Wert
pi = 3.141592653589793 # Fließkommazahl
message = 'And now for something completely different' # String
```

Verwendung von Variablen

- Variablenwerte können ausgegeben werden: `n` → 17
- Variablen in arithmetischen Ausdrücken: `n + 25` → 42
- Variablen in Funktionsaufrufen: `round(pi)` → 3
- Strings können mit `len()` gemessen werden: `len(message)` → 42

Regeln für Variablennamen

- Können beliebig lang sein
- Dürfen aus Buchstaben und Ziffern bestehen
- Dürfen nicht mit einer Ziffer beginnen
- Können Unterstriche enthalten (z.B. `your_name`, `air_speed_of_unladen_swallow`)
- Dürfen keine Sonderzeichen oder Leerzeichen enthalten
- Dürfen kein Schlüsselwort sein
- Sind normalerweise kleingeschrieben

Ungültige Variablennamen

- `76trombones` - beginnt mit einer Ziffer
- `million!` - enthält Sonderzeichen
- `class` - ist ein Schlüsselwort

Python-Schlüsselwörter (können nicht als Variablennamen verwendet werden)

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Zustandsdiagramme

- Visuelle Darstellung von Variablen und ihren Werten
- Beispiel:

```
n → 17
pi → 3.141592653589793
message → 'And now for something completely different'
```

Die import-Anweisung

Module importieren

- **Modul:** Eine Sammlung von Variablen und Funktionen
- `import math` lädt das math-Modul
- Nach dem Import kann auf Funktionen und Variablen des Moduls zugegriffen werden
- Syntax: `modulname.funktionsname()` oder `modulname.variablenname`
- Beispiel: `math.pi` gibt `3.141592653589793` zurück

Funktionen im math-Modul

- `math.sqrt(x)` - Quadratwurzel von x: `math.sqrt(25) → 5.0`
- `math.pow(x, y)` - x hoch y: `math.pow(5, 2) → 25.0`
- `math.sin(x)`, `math.cos(x)` - Trigonometrische Funktionen
- `math.exp(x)` - e hoch x (e^x)

Konstanten im math-Modul

- `math.pi` - Kreiszahl π (3.141592...)
- `math.e` - Eulersche Zahl e (2.718281...)

Punktoperator

- Der `.` (Punkt) zwischen Modulname und Funktion wird als **Punktoperator** bezeichnet
- Beispiel: `math.sqrt(25)` - `math` ist das Modul, `sqrt` ist die Funktion

Ausdrücke und Anweisungen

Ausdrücke

- Ein **Ausdruck** kann sein:
 - Ein einzelner Wert (Integer, Float, String)
 - Eine Kombination von Werten und Operatoren
 - Variablennamen oder Funktionsaufrufe
- Komplexe Ausdrücke: `19 + n + round(math.pi) * 2`
- **Auswertung:** Durchführung der Operationen in einem Ausdruck, um einen Wert zu berechnen

Anweisungen

- Eine **Anweisung** führt eine Aktion aus, hat aber keinen Wert
- Beispiele:
 - Zuweisungsanweisung: `n = 17`
 - Import-Anweisung: `import math`
- **Ausführung**: Die Befehle einer Anweisung befolgen

Die print-Funktion

Grundlegende Verwendung

- `print(wert)` gibt den Wert auf der Konsole aus
- `print('Text')` gibt Text aus
- Mehrere Ausdrücke ausgeben:

```
print(n+2)    # Ausgabe: 19
print(n+3)    # Ausgabe: 20
```

Mehrere Werte in einer Zeile ausgeben

- `print(ausdruck1, ausdruck2, ...)` gibt mehrere Werte aus, getrennt durch Leerzeichen
- Beispiel: `print('Value of pi is', math.pi)` → Value of pi is 3.141592653589793

Funktionsargumente

Was sind Argumente?

- **Argument**: Ein Wert, der einer Funktion bei ihrem Aufruf übergeben wird
- Argumente stehen in Klammern nach dem Funktionsnamen

Arten von Funktionsargumenten

- Funktionen mit einem Argument: `int('101') → 101`
- Funktionen mit zwei Argumenten: `math.pow(5, 2) → 25.0`
- Funktionen mit optionalen Argumenten:
 - `int('101', 2) → 5` (zweites Argument gibt die Basis an)
 - `round(math.pi, 3) → 3.142` (zweites Argument gibt die Anzahl der Nachkommastellen an)
- Funktionen mit variabler Anzahl von Argumenten: `print('Any', 'number', 'of', 'arguments')`

Fehler bei Argumenten

- Zu viele Argumente: `float('123.0', 2) → TypeError`
- Zu wenige Argumente: `math.pow(2) → TypeError`
- Falscher Argumenttyp: `math.sqrt('123') → TypeError`

Kommentare

Syntax und Verwendung

- Beginnen mit dem Doppelkreuz #
- Alles ab dem # bis zum Zeilenende wird vom Interpreter ignoriert
- Einfügen von Kommentaren:

```
# Dies ist ein Kommentar auf einer eigenen Zeile  
x = 5 # Dies ist ein Kommentar am Ende einer Codezeile
```

Gute Kommentare schreiben

- Erklären das "Warum", nicht das "Was"
- Schlechter Kommentar: `v = 8 # assign 8 to v` (unnötig, da offensichtlich)
- Guter Kommentar: `v = 8 # velocity in miles per hour` (gibt zusätzliche Information)
- Gute Variablennamen können Kommentare überflüssig machen

Fehlerarten und Debugging

Syntaxfehler

- Fehler in der Struktur des Programms
- Programm wird nicht ausgeführt
- Beispiele:
 - Ungültige Variablennamen: `million! = 1000000`
 - Verwendung von Schlüsselwörtern als Variablennamen: `class = 'Python'`

Laufzeitfehler (Ausnahmen)

- Treten während der Ausführung des Programms auf
- Führen zum Abbruch des Programms
- Beispiel: `'126' / 3` → `TypeError`, da ein String nicht durch eine Zahl geteilt werden kann

Semantische Fehler

- Programm läuft ohne Fehlermeldung, tut aber nicht das Gewünschte
- Beispiel: `1 + 3 / 2` ergibt `2.5` statt `2.0` (bei Berechnung des Durchschnitts von 1 und 3)
- Schwierig zu finden, erfordern logisches Denken

Übungsbeispiele

Beispiel 1: Kugelvolumen berechnen

```
import math  
  
# Radius in Zentimetern  
radius = 5  
  
# Volumen in Kubikzentimetern berechnen mit  $V = \frac{4}{3} * \pi * r^3$ 
```

```
volume = (4/3) * math.pi * radius**3

print("Das Volumen einer Kugel mit Radius", radius, "cm beträgt", volume, "cm³")
```

Beispiel 2: Trigonometrische Identität überprüfen

```
import math

x = 42 # Beliebiger Wert für x

# Berechnen der Summe von (cos x)² + (sin x)² (sollte 1 ergeben)
result = math.cos(x)**2 + math.sin(x)**2

print("(cos x)² + (sin x)² =", result) # Sollte ungefähr 1 sein
```

Beispiel 3: e^2 auf verschiedene Arten berechnen

```
import math

# Methode 1: Mit Potenzierungsoperator
result1 = math.e ** 2

# Methode 2: Mit math.pow
result2 = math.pow(math.e, 2)

# Methode 3: Mit math.exp
result3 = math.exp(2)

print("e² mit ** Operator:", result1)
print("e² mit math.pow():", result2)
print("e² mit math.exp():", result3)
```

Glossar

Begriff	Beschreibung
Variable	Ein Name, der auf einen Wert verweist
Zuweisungsanweisung	Eine Anweisung, die einer Variablen einen Wert zuweist
Zustandsdiagramm	Eine grafische Darstellung von Variablen und ihrer Werte
Schlüsselwort	Ein spezielles Wort für die Definition der Programmstruktur
import-Anweisung	Eine Anweisung, die eine Moduldatei lädt
Modul	Eine Sammlung von Variablen und Funktionen
Punktoperator	Der Operator (.) für den Zugriff auf Funktionen in einem Modul

Begriff	Beschreibung
Anweisung	Eine oder mehr Codezeilen, die für einen Befehl stehen
Auswertung	Durchführung der Operationen in einem Ausdruck, um einen Wert zu berechnen
Ausführung	Die Befehle einer Anweisung befolgen
Argument	Ein Wert, der einer Funktion bei ihrem Aufruf übergeben wird
Kommentar	Text im Programm, der vom Interpreter ignoriert wird
Laufzeitfehler	Ein Fehler, der während der Ausführung auftritt
Ausnahme	Ein Fehler, der während der Ausführung des Programms auftritt
Semantischer Fehler	Ein Fehler, der dafür sorgt, dass das Programm nicht das tut, was es soll

Python Funktionen

Funktionen definieren und aufrufen

Funktionsdefinition

Eine Funktion wird mit dem Schlüsselwort `def` definiert und besteht aus:

- **Header:** Erste Zeile mit dem Namen der Funktion und den Parametern in Klammern
- **Body:** Der eingerückte Code-Block (üblicherweise 4 Leerzeichen), der bei Funktionsaufruf ausgeführt wird

```
def funktionsname(parameter1, parameter2, ...):  
    # Funktionskörper  
    anweisung1  
    anweisung2  
    ...
```

Beispiel einer einfachen Funktion

```
def print_lyrics():  
    print("I'm a lumberjack, and I'm okay.")  
    print("I sleep all night and I work all day.")
```

Funktionsaufruf

Eine Funktion wird aufgerufen, indem man ihren Namen gefolgt von runden Klammern schreibt:

```
print_lyrics() # Ruft die Funktion auf und führt ihre Anweisungen aus
```

Funktionen mit Parametern

Parameter sind Variablen, die in der Funktionsdefinition deklariert und bei jedem Aufruf mit konkreten Werten (Argumenten) gefüllt werden:

```
def print_twice(string):  
    print(string)  
    print(string)  
  
# Aufruf mit einem String-Argument  
print_twice('Dennis Moore, ')  
  
# Aufruf mit einer Variable als Argument
```

```
line = 'Dennis Moore, '  
print_twice(line)
```

Mehrere Parameter

Funktionen können mehrere Parameter haben:

```
def repeat(word, n):  
    print(word * n)  
  
# Aufruf mit zwei Argumenten  
spam = 'Spam, '  
repeat(spam, 4) # Gibt "Spam, Spam, Spam, Spam, " aus
```

Funktionskomposition

Funktionen rufen andere Funktionen auf

Funktionen können andere Funktionen aufrufen, was die Wiederverwendung von Code ermöglicht:

```
def first_two_lines():  
    repeat(spam, 4)  
    repeat(spam, 4)  
  
def last_three_lines():  
    repeat(spam, 2)  
    print('(Lovely Spam, Wonderful Spam!)')  
    repeat(spam, 2)  
  
def print_verse():  
    first_two_lines()  
    last_three_lines()
```

Aufrufhierarchie

- Beim Aufruf einer Funktion wird deren Code ausgeführt
- Wenn eine Funktion eine andere aufruft, wird die Ausführung der ersten Funktion pausiert
- Nach Beendigung der aufgerufenen Funktion wird die Ausführung der ersten Funktion fortgesetzt

Wiederholung mit for-Schleifen

Grundstruktur einer for-Schleife

```
for variablenname in range(anzahl):  
    # Schleifenkörper  
    anweisung1
```

```
anweisung2  
...
```

Beispiel

```
for i in range(2):  
    print(i) # Gibt "0" und dann "1" aus
```

Mit Funktionen kombinieren

```
def print_n_verses(n):  
    for i in range(n):  
        print_verse()  
        print() # Leere Zeile zwischen den Versen
```

Wirkungsweise von range

- `range(2)` erzeugt die Sequenz: 0, 1
- `range(1, 5)` erzeugt die Sequenz: 1, 2, 3, 4
- `range(0, 10, 2)` erzeugt die Sequenz: 0, 2, 4, 6, 8
- `range(5, 0, -1)` erzeugt die Sequenz: 5, 4, 3, 2, 1

Variablen-Geltungsbereich (Scope)

Lokale Variablen

- Variablen, die innerhalb einer Funktion definiert werden, sind lokal zur Funktion
- Sie existieren nur während der Ausführung der Funktion
- Nach dem Ende der Funktion werden lokale Variablen gelöscht

```
def cat_twice(part1, part2):  
    cat = part1 + part2 # Lokale Variable 'cat'  
    print_twice(cat)  
  
# cat existiert hier nicht mehr  
# print(cat) # Würde einen NameError verursachen
```

Parameter sind lokal

Parameter funktionieren wie lokale Variablen und existieren nur innerhalb der Funktion:

```
def print_twice(string): # 'string' ist ein lokaler Parameter  
    print(string)
```

```
print(string)

# 'string' existiert hier nicht
```

Globale Variablen

Variablen, die außerhalb von Funktionen definiert werden, sind global und können innerhalb von Funktionen verwendet werden:

```
message = "Hello, World!" # Globale Variable

def greet():
    print(message) # Verwendet die globale Variable 'message'

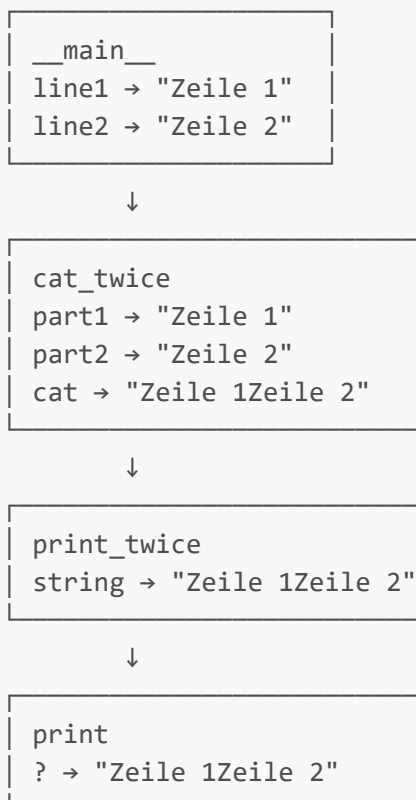
greet() # Gibt "Hello, World!" aus
```

Stack-Diagramme und Debugging

Stack-Diagramm

- Visuelle Darstellung der Funktionsaufrufe und ihrer Variablen
- Jede Funktion wird durch einen Frame (Kasten) dargestellt
- Der Frame enthält Parameter und lokale Variablen der Funktion

Beispiel:



Traceback

- Bei Laufzeitfehlern zeigt Python einen Traceback an
- Ein Traceback listet die aufgerufenen Funktionen in umgekehrter Reihenfolge auf
- Hilft bei der Identifikation, wo der Fehler aufgetreten ist

Beispiel für einen Traceback:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in cat_twice
  File "<stdin>", line 2, in print_twice
NameError: name 'cat' is not defined
```

Typische Fehler in Funktionen

NameError

- Entsteht, wenn auf eine Variable zugegriffen wird, die nicht definiert ist
- Häufig durch Tippfehler oder Zugriff auf lokale Variablen außerhalb ihres Geltungsbereichs

```
def print_twice(string):
    print(cat) # NameError: cat ist nicht definiert
    print(cat)
```

TypeError

- Entsteht, wenn eine Funktion mit falschen Argumenttypen aufgerufen wird
- Oder wenn die Anzahl der Argumente nicht mit der Anzahl der Parameter übereinstimmt

```
def multiply(a, b):
    return a * b

multiply(3) # TypeError: fehlendes Argument
multiply('text', 5.0, True) # TypeError: zu viele Argumente
```

SyntaxError

- Entsteht bei Fehlern in der Struktur des Codes
- Zum Beispiel fehlende Doppelpunkte, falsche Einrückung oder fehlende Klammern

```
def broken_function() # SyntaxError: fehlendes ':'
    print("Dies wird nie ausgeführt")
```

Warum Funktionen?

Vorteile von Funktionen

1. **Wiederverwendbarkeit:** Code muss nur einmal geschrieben werden
2. **Abstraktionsniveau:** Komplexe Operationen können einen einfachen Namen erhalten
3. **Modularität:** Programme können in kleinere, überschaubare Teile aufgeteilt werden
4. **Testbarkeit:** Einzelne Funktionen können unabhängig getestet werden
5. **Wartbarkeit:** Änderungen müssen nur an einer Stelle vorgenommen werden

Best Practices

- Eine Funktion sollte eine klar definierte Aufgabe erfüllen
- Wähle aussagekräftige Namen für Funktionen und Parameter
- Kommentiere komplexe Funktionen, um ihren Zweck zu erklären
- Vermeide zu lange Funktionen; teile sie in kleinere Funktionen auf

Übungsbeispiele

Beispiel 1: Text rechtsbündig ausgeben

```
def print_right(text):  
    # Berechne die Anzahl der benötigten Leerzeichen  
    spaces = 40 - len(text)  
    # Verwende den String-Multiplikationsoperator für Leerzeichen  
    print(' ' * spaces + text)  
  
print_right("Monty")           # Ausgabe: "                Monty"  
print_right("Python's")       # Ausgabe: "           Python's"  
print_right("Flying Circus")  # Ausgabe: "      Flying Circus"
```

Beispiel 2: Dreieck aus Zeichen erstellen

```
def triangle(char, height):  
    for i in range(height + 1):  
        print(char * i)  
  
triangle('L', 5)  
# Ausgabe:  
#  
# L  
# LL  
# LLL  
# LLLL  
# LLLLL
```

Beispiel 3: Rechteck aus Zeichen erstellen

```
def rectangle(char, width, height):
    for i in range(height):
        print(char * width)

rectangle('H', 5, 4)
# Ausgabe:
# HHHHH
# HHHHH
# HHHHH
# HHHHH
```

Beispiel 4: "99 Bottles of Beer"-Song

```
def bottle_verse(n):
    print(f"{n} bottles of beer on the wall")
    print(f"{n} bottles of beer ")
    print("Take one down, pass it around")
    print(f"{n-1} bottles of beer on the wall")

# Eine Strophe ausgeben
bottle_verse(99)

# Alle Strophen ausgeben
for n in range(99, 0, -1):
    bottle_verse(n)
    print() # Leere Zeile zwischen den Strophen
```

Glossar

Begriff	Beschreibung
Funktionsdefinition	Eine Anweisung, die eine Funktion erstellt (<code>def name():</code>)
Header	Die erste Zeile einer Funktionsdefinition
Body	Die Folge der Anweisungen innerhalb einer Funktionsdefinition
Funktionsobjekt	Ein Wert, der von einer Funktionsdefinition erzeugt wird
Parameter	Ein Name innerhalb einer Funktion, der auf einen übergebenen Wert verweist
Argument	Ein Wert, der einer Funktion beim Aufruf übergeben wird
Schleife	Eine Anweisung, die andere Anweisungen wiederholt ausführt
Lokale Variable	Eine innerhalb einer Funktion definierte Variable, nur dort verfügbar
Stack-Diagramm	Grafische Darstellung der Funktionsaufrufe und ihrer Variablen

Begriff	Beschreibung
Frame	Ein Kasten im Stack-Diagramm, der für einen Funktionsaufruf steht
Traceback	Liste der aufgerufenen Funktionen, die ausgegeben wird, wenn ein Fehler auftritt

Python Interfaces und Beispiele

Das jupyterturtle-Modul

Das jupyterturtle-Modul bietet eine einfache Möglichkeit, grafische Zeichnungen durch Programmierung zu erstellen. Es basiert auf dem Konzept einer "Schildkröte", die sich auf einer Zeichenfläche bewegt und dabei eine Spur hinterlässt.

Import und grundlegende Funktionen

```
# Vollständiger Import
import jupyterturtle

# Oder Import einzelner Funktionen
from jupyterturtle import make_turtle, forward, left, right, penup, pendown
```

Wichtige Befehle für die Schildkröte

Befehl	Beschreibung
<code>make_turtle()</code>	Erstellt eine Zeichenfläche und eine Schildkröte
<code>forward(length)</code>	Bewegt die Schildkröte vorwärts um <code>length</code> Einheiten
<code>left(angle)</code>	Dreht die Schildkröte um <code>angle</code> Grad nach links
<code>right(angle)</code>	Dreht die Schildkröte um <code>angle</code> Grad nach rechts
<code>penup()</code>	Hebt den Stift an (bewegt ohne zu zeichnen)
<code>pendown()</code>	Senkt den Stift ab (zeichnet beim Bewegen)

Beispiel: Ein einfaches Quadrat zeichnen

```
make_turtle()
forward(50)
left(90)
forward(50)
left(90)
forward(50)
left(90)
forward(50)
left(90)
```

Beispiel: Ein Quadrat mit einer Schleife zeichnen

```
make_turtle()
for i in range(4):
    forward(50)
    left(90)
```

Verkapselung und Verallgemeinerung

Verkapselung

Verkapselung bedeutet, eine Folge von Anweisungen in eine Funktion einzukapseln. Dies macht den Code übersichtlicher und wiederverwendbar.

```
def square():
    for i in range(4):
        forward(50)
        left(90)

# Aufruf der Funktion
make_turtle()
square()
```

Verallgemeinerung

Verallgemeinerung bedeutet, konkrete Werte durch Parameter zu ersetzen, um die Funktion flexibler zu machen.

```
def square(length):
    for i in range(4):
        forward(length)
        left(90)

# Aufruf mit verschiedenen Größen
make_turtle()
square(30) # Kleines Quadrat
square(60) # Größeres Quadrat
```

Weitere Verallgemeinerung: Ein Polygon zeichnen

```
def polygon(n, length):
    angle = 360 / n
    for i in range(n):
        forward(length)
        left(angle)

# Beispielaufrufe
```

```
make_turtle()
polygon(3, 50) # Dreieck
polygon(5, 30) # Fünfeck
polygon(8, 20) # Achteck
```

Schlüsselwortargumente

Schlüsselwortargumente machen den Code lesbarer, besonders bei Funktionen mit mehreren Parametern.

```
# Ohne Schlüsselwortargumente
polygon(7, 30)

# Mit Schlüsselwortargumenten
polygon(n=7, length=30) # Deutlicher, wofür die Werte stehen
```

Kreis- und Bogenzeichnung

Kreis zeichnen durch Näherung mit einem Polygon

```
import math

def circle(radius):
    circumference = 2 * math.pi * radius
    n = 30 # Anzahl der Segmente
    length = circumference / n
    polygon(n, length)
```

Refaktorisierung mit polyline

```
def polyline(n, length, angle):
    """Zeichnet n Liniensegmente mit gegebener Länge und Winkel.

    n: Anzahl der Segmente (Integer)
    length: Länge jedes Segments
    angle: Winkel zwischen den Segmenten (in Grad)
    """
    for i in range(n):
        forward(length)
        left(angle)
```

Polygon mit polyline implementieren

```
def polygon(n, length):
    """Zeichnet ein regelmäßiges n-Eck mit Seitenlänge length."""
```

```
angle = 360.0 / n
polyline(n, length, angle)
```

Bogen zeichnen

```
def arc(radius, angle):
    """Zeichnet einen Bogen mit gegebenem Radius und Winkel."""
    arc_length = 2 * math.pi * radius * angle / 360
    n = 30 # Anzahl der Segmente
    length = arc_length / n
    step_angle = angle / n
    polyline(n, length, step_angle)
```

Kreis mit arc implementieren

```
def circle(radius):
    """Zeichnet einen Kreis mit gegebenem Radius."""
    arc(radius, 360)
```

Funktionsdesign und Interface

Interface vs. Implementierung

- **Interface:** Beschreibt, wie die Funktion verwendet wird (Name, Parameter, Aufgabe)
- **Implementierung:** Beschreibt, wie die Funktion intern arbeitet

Beispiel für unterschiedliche Implementierungen mit gleichem Interface

```
# Erste Implementierung von circle
def circle(radius):
    circumference = 2 * math.pi * radius
    n = 30
    length = circumference / n
    polygon(n, length)

# Zweite Implementierung (nach Refaktorisierung)
def circle(radius):
    arc(radius, 360)
```

Beide Funktionen haben das gleiche Interface (Name und Parameter), aber unterschiedliche Implementierungen.

Docstrings

Docstrings sind Zeichenketten am Anfang einer Funktion, die das Interface dokumentieren.


```
def polyline(n, length, angle):
    """Zeichnet Liniensegmente mit gegebener Länge und Winkel.

    n: Anzahl der Liniensegmente (ganzzahlig)
    length: Länge der Liniensegmente
    angle: Winkel zwischen den Segmenten (in Grad)
    """
    for i in range(n):
        forward(length)
        left(angle)
```

Konventionen für Docstrings

- Werden mit dreifachen doppelten Anführungszeichen umgeben ("""")
- Enthalten eine kurze Beschreibung der Funktion
- Erklären die Parameter und deren Typ
- Beschreiben ggf. den Rückgabewert und wichtige Hinweise

Prä- und Postkonditionen

- **Präkonditionen:** Bedingungen, die vor der Ausführung der Funktion erfüllt sein müssen
 - Beispiel: `n` muss ein Integer sein, `length` eine positive Zahl
 - Die Verantwortung liegt beim Aufrufer
- **Postkonditionen:** Bedingungen, die nach der Ausführung der Funktion gelten
 - Beispiel: Eine bestimmte Form wurde gezeichnet, die Schildkröte hat ihre Position geändert
 - Die Verantwortung liegt bei der Funktion selbst

Entwicklungsplan: Verkapselung und Verallgemeinerung

1. Beginnen Sie mit einem kleinen, funktionierenden Programm ohne Funktionen
2. Identifizieren Sie zusammengehörige Codeblöcke und kapseln Sie diese in Funktionen ein
3. Verallgemeinern Sie die Funktionen durch Hinzufügen geeigneter Parameter
4. Wiederholen Sie die Schritte 1-3, bis Sie einen Satz fehlerfreier Funktionen haben
5. Verbessern Sie das Programm durch Refaktorisierung

Refaktorisierung

Refaktorisierung bedeutet, funktionierenden Code umzugestalten, um ihn zu verbessern, ohne sein Verhalten zu ändern.

Beispiel für Refaktorisierung

```
# Vor der Refaktorisierung: Ähnlicher Code in mehreren Funktionen

def draw_triangle(side):
```

```

    for i in range(3):
        forward(side)
        left(120)

def draw_square(side):
    for i in range(4):
        forward(side)
        left(90)

# Nach der Refaktorisierung: Gemeinsame Struktur extrahiert

def polygon(n, side):
    angle = 360 / n
    for i in range(n):
        forward(side)
        left(angle)

def draw_triangle(side):
    polygon(3, side)

def draw_square(side):
    polygon(4, side)

```

Praktische Hilfsfunktionen

Bewegung ohne Zeichnen (Jump)

```

def jump(length):
    """Bewegt die Schildkröte vorwärts, ohne eine Spur zu hinterlassen.

    Postbedingung: Stift bleibt unten.
    """
    penup()
    forward(length)
    pendown()

```

Nützliche Tipps

1. **Beschleunigung der Zeichnung:** `make_turtle(delay=0.02)` setzt eine kürzere Verzögerung zwischen den Zeichenschritten
2. **Verschachtelte Funktionsaufrufe:** Komplexe Formen können durch Kombination einfacherer Funktionen erstellt werden
3. **Wiederholung von Formen:** Verwenden Sie Schleifen, um Formen zu wiederholen oder Muster zu erzeugen
4. **Nachvollziehbarkeit:** Stack-Diagramme helfen, den Ablauf verschachtelter Funktionsaufrufe zu verstehen

Stack-Diagramm für verschachtelte Funktionsaufrufe

```
circle  
radius = 30
```

↓

```
arc  
radius = 30  
angle = 360
```

↓

```
polyline  
n = 60  
length = 3.04  
angle = 5.8
```

Glossar

Begriff	Bedeutung
Zeichenfläche (Canvas)	Ein Bereich auf dem Bildschirm für grafische Darstellungen
Verkapselung	Prozess, eine Anweisungsfolge in eine Funktion zu verwandeln
Verallgemeinerung	Ersetzung spezifischer Werte durch Parameter
Schlüsselwortargument	Argument, das den Namen des Parameters enthält
Refaktorisierung	Umgestalten von Code bei gleichbleibendem Verhalten
Entwicklungsplan	Systematische Vorgehensweise zum Schreiben von Programmen
Docstring	Dokumentationstext am Anfang einer Funktion
Multiline-String	String, der sich über mehrere Zeilen erstreckt
Präkondition	Bedingung, die vor der Funktionsausführung erfüllt sein muss
Postkondition	Bedingung, die nach der Funktionsausführung gelten muss
Interface	Beschreibung, wie eine Funktion verwendet wird
Implementierung	Details, wie eine Funktion intern arbeitet